

Tout ce qu'il faut comprendre

Runtrack 01 — VM & Docker — Leon Camuzat

Ce document résume tous les concepts fondamentaux abordés dans le projet. Pour chaque notion, tu trouveras une définition claire, son rôle dans le projet, et les commandes associées. L'objectif est de pouvoir expliquer chaque concept de tête.

01	Virtualisation & VM	VMware, Debian, machine hôte/invitée
02	Docker — Architecture	Daemon, client, images, conteneurs
03	Images Docker	Dockerfile, layers, build, FROM/RUN/CMD
04	Conteneurs	run, ps, rm, exec, ports, cycle de vie
05	Volumes & Persistance	Volume nommé, montage, partage
06	Docker Compose	YAML, services, dépendances, réseaux
07	Registry	Docker Hub, registry local, tag, push, pull
08	Dockerfile — Instructions clés	FROM, RUN, CMD, EXPOSE, COPY, ARG, ENV
09	Réseau Docker	Ports, bridge, host, NAT, -p
10	Checklist — Ce que tu dois savoir	Questions types d'évaluation

01 VIRTUALISATION & MACHINE VIRTUELLE

Machine Virtuelle (VM)

Définition : Ordinateur simulé par logiciel qui tourne sur un ordinateur réel (l'hôte). Elle a son propre OS, ses ressources (CPU, RAM, disque) allouées, et est complètement isolée.

Pourquoi c'est important : Dans ce projet, toute l'installation Docker se fait dans une VM Debian 13 sur VMware Workstation. Sans VM, on risquerait de casser sa machine principale.

Machine hôte

Le PC physique sur lequel VMware est installé. Il héberge la VM et partage ses ressources.

Machine invitée (guest)

La VM elle-même. C'est Debian 13 dans ce projet. Elle croit être un vrai ordinateur.

Hyperviseur

Logiciel qui crée et gère les VMs. VMware Workstation est un hyperviseur de type 2 (tourne sur un OS existant).

Snapshot

Photo de l'état de la VM à un instant T. Permet de revenir en arrière si on casse quelque chose.

■ **VM vs Conteneur :** une VM virtualise tout le matériel + un OS complet (lourd, ~minutes pour démarrer). Un conteneur partage le noyau de l'hôte, isole juste le processus (léger, ~secondes).

02 DOCKER — ARCHITECTURE

Docker

Définition : Plateforme de containerisation qui permet de packager une application avec toutes ses dépendances dans un conteneur portable et isolé.

Pourquoi c'est important : Docker garantit que l'application fonctionne de la même façon partout : en développement, en test, en production.

Composant	Rôle	Analogie
Docker Daemon (dockerd)	Le serveur Docker. Tourne en arrière-plan, gère images, conteneurs, réseaux, volumes.	<i>Le moteur d'une voiture</i>
Docker Client (CLI)	L'outil en ligne de commande. Envoie des commandes au daemon via l'API.	<i>Le volant de la voiture</i>
Docker Hub / Registry	Dépôt d'images. Docker Hub est le registry public officiel.	<i>App Store des images</i>
Image	Template immuable contenant OS + app + dépendances. Ne change jamais.	<i>La recette de cuisine</i>
Conteneur	Instance en cours d'exécution d'une image. Isolé, éphémère par défaut.	<i>Le plat cuisiné</i>

■■ **docker.sock** : fichier socket Unix `/var/run/docker.sock`. C'est le canal de communication entre le client Docker et le daemon. Monter ce fichier dans Portainer lui donne le contrôle total de Docker.

03 IMAGES DOCKER

Image Docker

Définition : Fichier en lecture seule contenant le système de fichiers complet d'un conteneur : OS de base, bibliothèques, application, configuration. Construite en couches (layers) superposées.

Pourquoi c'est important : C'est le point de départ de tout conteneur. Dans ce projet on a créé : *my-hello* (message custom), *my-ssh* (serveur SSH), *my-nginx* (serveur web).

Le système de layers (couches) :

Chaque instruction du Dockerfile crée une nouvelle couche. Les couches sont mises en cache : si rien ne change, Docker réutilise le cache → builds plus rapides.

Couche 4	<code>CMD ["echo", "Hello"]</code>	Commande de démarrage
Couche 3	<code>RUN apt install -y nginx</code>	Installation de nginx
Couche 2	<code>RUN apt update</code>	Mise à jour des paquets
Couche 1 (base)	<code>FROM debian:stable-slim</code>	Image de base Debian

Commandes images :

```
docker build -t mon-image:v1 . # construire depuis le Dockerfile du dossier courant
docker images # lister toutes les images locales
docker rmi mon-image # supprimer une image
docker pull nginx:alpine # télécharger une image depuis Docker Hub
docker tag mon-image registry:5000/mon-image:v1 # renommer/étiqueter une image
docker push registry:5000/mon-image:v1 # envoyer vers un registry
```

04 CONTENEURS — CYCLE DE VIE

Conteneur Docker

Définition : Processus isolé sur l'hôte, créé depuis une image. Il a son propre système de fichiers, ses variables d'environnement, et ses interfaces réseau. Par défaut, son filesystem est éphémère.

Pourquoi c'est important : C'est l'unité d'exécution de Docker. On peut en créer des dizaines depuis la même image. Exemple : `ssh-container`, `nginx-container`, `ftp`, `registry`...

État	Signification	Commande
created	Créé mais pas démarré	<code>docker create</code>
running	En cours d'exécution	<code>docker run</code> / <code>docker start</code>
paused	Suspendu (processus gelé)	<code>docker pause</code>
stopped	Arrêté (processus terminé)	<code>docker stop</code>
removed	Supprimé définitivement	<code>docker rm</code>

Commandes essentielles :

```
docker run -d -p 8080:80 --name mon-conteneur nginx
# -d : détaché (arrière-plan) -p hôte:conteneur --name : nom custom

docker exec -it mon-conteneur bash # entrer dans le conteneur en interactif
docker logs -f mon-conteneur # suivre les logs en temps réel
docker inspect mon-conteneur # infos détaillées (IP, volumes, config...)
docker stop mon-conteneur # arrêter proprement (SIGTERM)
docker rm -f mon-conteneur # forcer la suppression
docker ps -a # voir tous les conteneurs avec leur état
```

■ **--rm** dans `docker run` : supprime automatiquement le conteneur quand il s'arrête. Parfait pour les tests. Exemple :

```
docker run -it --rm debian bash
```

05 VOLUMES & PERSISTANCE DES DONNÉES

Volume Docker

Définition : Espace de stockage géré par Docker, indépendant du cycle de vie des conteneurs. Les données écrites dans un volume survivent à la suppression du conteneur.

Pourquoi c'est important : Sans volume, toutes les données créées dans un conteneur sont perdues quand il est supprimé. Dans le projet JOB06 : le volume 'partage' permet à c1 d'écrire et à c2 de lire.

Volume nommé

Créé avec `docker volume create nom`. Stocké dans `/var/lib/docker/volumes/`. Géré entièrement par Docker.

Bind mount

Monte un dossier de l'hôte dans le conteneur. Syntaxe : `-v /chemin/hote:/chemin/conteneur`. Utilisé pour Portainer avec `docker.sock`.

```
docker volume create partage # créer un volume nommé
docker volume ls # lister les volumes
docker volume inspect partage # voir les détails (chemin, options)
docker volume rm partage # supprimer un volume
docker volume prune # supprimer tous les volumes inutilisés

# Monter un volume dans un conteneur
docker run -v partage:/data debian # volume nommé -> /data dans le conteneur
docker run -v /home/leon:/app nginx # bind mount : dossier hôte -> /app
```

■ Dans `docker-compose.yml`, les volumes définis dans la section **volumes:** au niveau racine sont des volumes nommés partagés entre services. Dans JOB07, `nginx` et `ftp` partagent **webdata** : FTP dépose les fichiers, `nginx` les sert.

06 DOCKER COMPOSE

Docker Compose

Définition : Outil permettant de définir et gérer des applications multi-conteneurs dans un fichier YAML. Un seul fichier remplace plusieurs commandes docker run.

Pourquoi c'est important : Dans JOB07 : au lieu de lancer nginx et vsftpd séparément avec des options complexes, un docker-compose.yml décrit tout. Dans JOB09 : registry + registry-ui ensemble.

Structure d'un docker-compose.yml :

```
services: # liste des conteneurs à créer
  nom_service: # nom libre (devient aussi le hostname réseau)
  image: nginx:alpine # image à utiliser (depuis Docker Hub ou local)
  container_name: nginx # nom du conteneur (optionnel)
  ports:
    - "8080:80" # port_hôte:port_conteneur
  volumes:
    - webdata:/app # volume_nommé:chemin_dans_conteneur
  environment:
    - MA_VAR=valeur # variables d'environnement
  depends_on:
    - autre_service # attend que l'autre service soit démarré
  restart: always # redémarrer automatiquement si crash
  volumes: # déclarer les volumes nommés utilisés
  webdata:
```

Commandes Compose :

```
docker compose up # démarrer (voir les logs)
docker compose up -d # démarrer en arrière-plan
docker compose down # arrêter et supprimer les conteneurs
docker compose down -v # aussi supprimer les volumes
docker compose ps # état des services
docker compose logs -f # logs de tous les services
docker compose restart # redémarrer tous les services
```

07 REGISTRY DOCKER

Registry Docker

Définition : Serveur de stockage et distribution d'images Docker. Docker Hub est le registry public officiel. On peut déployer son propre registry privé (JOB09 : image registry:2).

Pourquoi c'est important : Permet de partager ses images entre machines, sauvegarder ses builds, et déployer ses apps sans dépendre de Docker Hub.

Docker Hub

Registry public officiel. Gratuit pour les images publiques. C'est là que viennent debian, nginx, python, etc. URL : hub.docker.com

Registry local (JOB09)

Image `registry:2` déployée en local. Accessible sur `localhost:5000`. Aucune authentification par défaut.

```
# Workflow complet : build → tag → push → pull

docker build -t my-nginx . # construire l'image
docker tag my-nginx localhost:5000/my-nginx:v1 # étiqueter pour le registry local
docker push localhost:5000/my-nginx:v1 # envoyer vers le registry

# Vérifier le contenu du registry
curl http://localhost:5000/v2/_catalog # {"repositories":["my-nginx"]}
curl http://localhost:5000/v2/my-nginx/tags/list # voir les tags disponibles

# Depuis une autre machine
docker pull 192.168.163.129:5000/my-nginx:v1
```

■ ■ **Tag latest** : quand tu fais `docker pull nginx` sans préciser de version, Docker télécharge le tag **latest**. En production, toujours spécifier une version précise : `nginx:1.25.3`

08 DOCKERFILE — INSTRUCTIONS CLÉS

Dockerfile

Définition : Fichier texte contenant les instructions pour construire une image Docker étape par étape. Chaque instruction crée une nouvelle couche dans l'image.

Pourquoi c'est important : C'est le 'code source' de ton image. Sans Dockerfile, tu ne peux pas reproduire ou partager ta façon de construire une image.

Instruction	Rôle	Exemple du projet
FROM	Image de base — obligatoire en 1er	FROM debian:stable-slim
RUN	Exécute une commande pendant le build (crée un layer)	RUN apt install -y openssh-server
CMD	Commande par défaut au démarrage du conteneur	CMD ["/usr/sbin/sshd", "-D"]
EXPOSE	Documente le port utilisé (ne l'ouvre pas sur l'hôte)	EXPOSE 22
COPY	Copie des fichiers de l'hôte vers l'image	COPY index.html /var/www/html/
ENV	Définit une variable d'environnement dans l'image	ENV NODE_ENV=production
ARG	Variable disponible pendant le build seulement	ARG VERSION=1.0
WORKDIR	Définit le répertoire de travail courant	WORKDIR /app
ENTRYPOINT	Commande principale (CMD lui passe des arguments)	ENTRYPOINT ["nginx"]

■ **CMD vs ENTRYPOINT :** CMD peut être écrasé par `docker run mon-image autre-commande`. ENTRYPOINT est fixe. Dans ce projet, CMD suffit pour tous les usages.

09 RÉSEAU DOCKER — PORTS & COMMUNICATION

Réseau Docker

Définition : Docker crée des réseaux virtuels pour permettre aux conteneurs de communiquer entre eux et avec l'extérieur. Chaque conteneur dans un réseau peut en joindre un autre par son nom de service.

Pourquoi c'est important : Dans JOB07, `nginx` et `ftp` sont dans le même réseau `Compose`. `nginx` peut joindre `ftp` par son nom. Dans JOB09, `registry-ui` atteint `registry` via `http://registry:5000`.

Port mapping (-p)

Relie un port de l'hôte à un port du conteneur. Format :

-p PORT_HÔTE:PORT_CONTENEUR

Exemples : `-p 8080:80` (`nginx`), `-p 2222:22` (`SSH`), `-p 5000:5000` (`registry`)

Réseau bridge (défaut)

Réseau virtuel privé créé par Docker. Les conteneurs communiquent entre eux. Docker Compose crée automatiquement un réseau bridge par projet, avec DNS interne (nom du service = `hostname`).

```
# Voir les réseaux Docker
docker network ls

# Inspecter un réseau (voir quels conteneurs y sont connectés)
docker network inspect bridge

# Portainer monte docker.sock pour accéder à l'API Docker
docker run -v /var/run/docker.sock:/var/run/docker.sock portainer/portainer-ce

# Exemple de communication inter-conteneurs dans Compose
# nginx peut accéder à ftp via http://ftp:21
# registry-ui accède à registry via http://registry:5000
```

■ ■ **PASV (FTP Passif)** dans JOB07 : en mode passif, le serveur FTP ouvre un port de données aléatoire. On fixe la plage 30000-30009 avec `PASV_MIN/MAX_PORT` pour pouvoir les exposer avec `-p`.

10 CHECKLIST — CE QUE TU DOIS SAVOIR EXPLIQUER

Questions types d'évaluation :

■	VM vs Conteneur	Quelle est la différence fondamentale ? Pourquoi utiliser l'un plutôt que l'autre ?
■	Image vs Conteneur	Quelle est la relation entre les deux ? Peut-on créer plusieurs conteneurs depuis la même image ?
■	Dockerfile	À quoi sert FROM, RUN, CMD, EXPOSE ? Dans quel ordre s'exécutent-ils ?
■	docker build	Que fait le point '.' à la fin ? Que fait -t ? Qu'est-ce qu'un layer ?
■	docker run	Expliquer -d, -p, --name, -v, -it, --rm et quand les utiliser
■	Volumes	Pourquoi les données sont perdues sans volume ? Différence bind mount / volume nommé ?
■	Docker Compose	À quoi sert depends_on ? Comment nginx et ftp partagent-ils webdata ?
■	Ports	Expliquer -p 8080:80 : qui est l'hôte, qui est le conteneur ?
■	Registry	Qu'est-ce que Docker Hub ? Comment push une image vers un registry local ?
■	docker.sock	Pourquoi Portainer monte-t-il ce fichier ? Quel risque de sécurité ?
■	restart: always	Quand est-ce utile ? Que se passe-t-il au reboot de la VM ?
■	daemon off	Pourquoi nginx a besoin de 'daemon off' dans CMD quand il tourne dans Docker ?

Commandes de survie :

```
# Tout arrêter et nettoyer
docker stop $(docker ps -q) && docker rm $(docker ps -aq)
docker system prune -af --volumes # ATTENTION : supprime tout !

# Diagnostiquer un conteneur qui plante
docker logs nom_conteneur # voir pourquoi il plante
docker inspect nom_conteneur # voir la config complète

# Entrer dans un conteneur en cours
docker exec -it nom_conteneur bash

# Voir ce qui écoute sur les ports
docker ps --format 'table {{.Names}}\t{{.Ports}}'
```

Résumé en une phrase : Docker permet de packager une app dans une **image** immuable, de l'exécuter dans un **conteneur** isolé, de persister les données dans des **volumes**, d'orchestrer plusieurs services avec **Compose**, et de partager les images via un **registry**.